Naive Bayes & K-Means Lecture Notes

Lecture 1: Naive Bayes

The Starting Point of Probabilistic Modeling

1. Start from the Classification Problem

Given an input x, the output is a class label y:

$$y \in 1, 2, \ldots, K$$

The goal is:

$$\hat{y} = \arg\max_{y} P(y|x)$$

This is the common objective of all classification models.

2. Bayes' Rule (Core Formula)

By Bayes' theorem:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

Since P(x) does not depend on y:

$$\hat{y} = \arg\max_{y} P(x|y)P(y)$$

• Discriminative models: directly model P(y|x)
• Generative models: model P(x|y) and P(y)

Naive Bayes is a generative model.

3. Conditional Independence Assumption (Key Assumption)

Let:

$$x = (x_1, x_2, \ldots, x_d)$$

Naive Bayes assumes conditional independence:

$$P(x|y) = \prod_{i=1}^{d} P(x_i|y)$$

Three key points to emphasize:

1. This assumption is almost certainly wrong.
2. But it dramatically reduces model complexity.
3. It makes learning computationally feasible.

This illustrates an important idea in machine learning:

Modeling assumptions ≠ the real world.

4. Parameter Learning (Discrete Features)

For each class y, we estimate:

$$P(x_i = v|y)$$

Using Maximum Likelihood Estimation (MLE):

$$P(x_i = v|y) = \frac{\text{count}(x_i = v, y)}{\text{count}(y)}$$

Laplace smoothing is often used:

$$P(x_i = v|y) = \frac{n_{iv} + 1}{n_y + |V|}$$

5. Common Variants of Naive Bayes

1. Bernoulli Naive Bayes
   • Features are binary (0/1)
   • Whether a feature appears
2. Multinomial Naive Bayes
   • Uses word counts / frequencies
   • Most common for text classification
3. Gaussian Naive Bayes
   • Continuous features

• Assumes:

$$x_i|y \sim \mathcal{N}(\mu_{iy}, \sigma_{iy}^2)$$

6. Decision Function (Log-Space)

To avoid numerical underflow, we use log-space:

$$\log P(y|x) \propto \log P(y) + \sum_i \log P(x_i|y)$$

7. Advantages and Disadvantages (Class Discussion)

Advantages
• Extremely fast
• Very few parameters
• Robust for small datasets
• Clear theoretical foundation

Disadvantages
• Independence assumption is too strong
• Limited expressive power
• Cannot model complex feature interactions

8. Summary

Naive Bayes = using a very strong independence assumption to turn an otherwise intractable probabilistic problem into one that can be solved by simple counting.

I. What Problem Does Naive Bayes Solve?
We want to classify:
• Input: x = (x_1, x_2, \dots, x_d)
• Output: class label y

Goal:

$$\hat{y} = \arg\max_y P(y|x)$$

II. Why Is Directly Modeling P(y|x) Hard?
Curse of dimensionality: estimating a high-dimensional distribution requires far more data than we usually have.

III. Bayes' Rule as a Change of Perspective

We rewrite:

$$\hat{y} = \arg\max_y P(x|y)P(y)$$

We shift from:

• "Given x, which class y is it?"

to

• "If y were known, what kind of x would it generate?"

This is the core idea of a generative model.

IV. The Real Difficulty: P(x|y) Is Still Intractable

$$P(x|y) = P(x_1, x_2, \ldots, x_d|y)$$

Even for modest d, the number of parameters explodes.

V. The "Fatal" Assumption (But Also the Key)

$$P(x|y) = \prod_{i=1}^{d} P(x_i|y)$$

Emphasize:

1. almost surely false
2. makes the model feasible
3. works surprisingly well

VI. Why Does It Still Work (Intuition)?

In text classification, words are correlated in reality, but NB assumes independence.
It can still work because classification often only needs relative scores, not perfectly calibrated probabilities, and errors may cancel across classes.

VII. How Are Parameters Learned?

Prior probability (counting):

$$P(y = k) = \frac{\text{count}(y = k)}{N}$$

Conditional probability (still counting), Bernoulli NB:

$$P(x_i = 1|y) = \frac{\text{count}(x_i = 1, y) + 1}{\text{count}(y) + 2}$$

(Laplace smoothing prevents zero probabilities.)
Learning is counting + normalization; no gradient descent.

VIII. Why Use Log at Prediction Time?

$$P(y|x) \propto P(y) \prod_i P(x_i|y)$$

Products underflow, so we use logs:

$$\log P(y|x) = \log P(y) + \sum_i \log P(x_i|y)$$

IX. What Is Naive Bayes Really Doing?

• Each feature provides a “vote” (log-likelihood contribution)
• Sum votes across features
• Choose the class with the highest score

This is structurally similar to Logistic Regression, MLPs, and linear classifiers.

Lecture 2: k-Means Clustering

The Starting Point of Unsupervised Learning and Representation Learning

1. Clustering Problem Definition

Given data:

$$x_1, x_2, \ldots, x_N, \quad x_i \in \mathbb{R}^d$$

Goal: partition the dataset into K clusters.

2. k-Means Objective Function

$$\min_{c_i, \mu_k} \sum_{i=1}^{N} |x_i - \mu_{c_i}|^2$$

This is a geometric objective, not a probabilistic model.

Where:
• x_i: the i-th sample
• $c_i \in \{1, \ldots, K\}$: cluster assignment
• μ_k: centroid of cluster k

k-means is not “guessing clusters”; it is optimizing a clear objective.

3. Lloyd’s Algorithm (Core Algorithm)

- **Step 1: Assignment**

$$c_i = \arg\min_k |x_i - \mu_k|^2$$

- **Step 2: Update**

$$\mu_k = \frac{1}{|\mathcal{C}k|} \sum i : c_i = k x_i$$

• Each step decreases the objective
• But the overall problem is non-convex

4. Why Does k-Means Converge?

Because:
• The assignment step does not increase the objective
• The update step is optimal given current assignments (the mean)

However:
• It only guarantees convergence to a local optimum.

5. Why Squared Euclidean Distance?

Two key reasons:

1. Simple geometric meaning
   • Euclidean distance = straight-line distance
   • Squaring penalizes far-away points more
2. Mathematically convenient
   For a fixed cluster C_k, minimizing

$$\sum_{x \in C_k} |x - \mu|^2$$

gives the closed-form solution

$$\mu = \frac{1}{|C_k|} \sum_{x \in C_k} x$$

That is why the centroid is the mean—this is derived, not arbitrary.

6. Initialization Sensitivity

k-means is extremely sensitive to initialization because:
• the objective is non-convex
• there are many local minima

Solutions:
• multiple random restarts
• k-means++

k-means++ intuition:
• first centroid random
• later centroids sampled proportional to distance from existing centroids
Goal: spread initial centroids apart.

7. Advantages and Disadvantages

Advantages
• simple and intuitive
• fast
• interpretable
• excellent for teaching

Disadvantages
• must choose K in advance
• sensitive to initialization
• sensitive to outliers(异常点)

8. KMeans Summary

k-means teaches us:

If you clearly specify what kind of clusters you want, the algorithm can be very simple.
But if real data violates those assumptions, simple algorithms will fail.

9. computation example

下面给你一个非常适合课堂板书/投影演示的 k-means 计算例子：数据点少、计算量小、两轮迭代就能看到“中心移动 + 分簇稳定”。

———

课堂演示例子：k=2 的 k-means 手算

1. 数据集（6 个二维点）

我们有 6 个点，明显分成左右两团：
• A: (1,1)
• B: (1,2)
• C: (2,1)
• D: (8,8)
• E: (9,8)
• F: (8,9)

目标：聚成 K=2 个簇。

———

1. 初始化（课堂可以“故意选得不完美”）

初始化两个中心（centroids）：
• \mu_1^{(0)} = (1,1)（选 A）
• \mu_2^{(0)} = (9,8)（选 E）

———

1. 第 1 轮：Assignment step（分配）

分配规则：

$$c_i = \arg\min_k |x_i - \mu_k|^2$$

为了手算方便，我们用平方距离（不用开根号），因为比较大小不变。

平方欧氏距离：

$$|(x,y)-(a,b)|^2=(x-a)^2+(y-b)^2$$

———

计算每个点到两中心的距离

点 A = (1,1)
• 到 \mu_1=(1,1)：
0
• 到 \mu_2=(9,8)：
(1-9)$^{2+(1-8)}$2=64+49=113

→ A 属于簇 1

———

点 B = (1,2)
• 到 \mu_1： (0)$^{2+(1)}$2=1
• 到 \mu_2： (1-9)$^{2+(2-8)}$2=64+36=100

→ B 属于簇 1

———

点 C = (2,1)
• 到 \mu_1： (1)$^{2+0}$2=1
• 到 \mu_2： (2-9)$^{2+(1-8)}$2=49+49=98

→ C 属于簇 1

———

点 D = (8,8)
• 到 \mu_1： (8-1)$^{2+(8-1)}$2=49+49=98
• 到 \mu_2： (8-9)$^{2+(8-8)}$2=1+0=1

→ D 属于簇 2

———

点 E = (9,8)
• 到 \mu_1： (8)$^{2+(7)}$2=64+49=113
• 到 \mu_2： 0

→ E 属于簇 2

———

点 F = (8,9)

• 到 \mu_1： (7)$^{2+(8)}$2=49+64=113

• 到 \mu_2： (8-9)$^{2+(9-8)}$2=1+1=2

→ F 属于簇 2

———

第 1 轮分簇结果

• 簇 1：A(1,1), B(1,2), C(2,1)

• 簇 2：D(8,8), E(9,8), F(8,9)

———

1. 第 1 轮：Update step（更新中心）

更新公式：

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

———

更新 \mu_1

簇 1 三个点：

• x 坐标均值：\frac{1+1+2}{3}=\frac{4}{3}

• y 坐标均值：\frac{1+2+1}{3}=\frac{4}{3}

所以：

$$\mu_1^{(1)} = \left(\frac{4}{3}, \frac{4}{3} \right) \approx (1.33, 1.33)$$

———

更新 \mu_2

簇 2 三个点：

• x 均值：\frac{8+9+8}{3}=\frac{25}{3}\approx 8.33

• y 均值：\frac{8+8+9}{3}=\frac{25}{3}\approx 8.33

所以：

$$\mu_2^{(1)} = \left(\frac{25}{3}, \frac{25}{3}\right) \approx (8.33, 8.33)$$

———

1. 第 2 轮：再次 Assignment（检查是否改变）

现在中心是：
• \mu_1^{(1)}\approx(1.33,1.33)
• \mu_2^{(1)}\approx(8.33,8.33)

直觉上：
• 左边三个点离 \mu_1 肯定更近
• 右边三个点离 \mu_2 肯定更近

你可以挑一个“边界点”快速验证，例如 C(2,1)：
• 到 \mu_1：(2-1.33)$^{2+(1-1.33)}$2\approx0.44+0.11=0.55
• 到 \mu_2：很大（约 100 级别）

不会换簇。

因此分配不变 → k-means 收敛。

非常好，这个 “坏初始化”例子在课堂上效果非常好，能让学生一下子明白：

k-means 不傻，但它真的很“看第一眼”

下面我给你一个可以直接板书 / PPT 演示的完整例子，逻辑清晰，而且第一轮一定“出事”。

———

课堂演示例子：k-means 的坏初始化（k=2）

1️⃣ 数据集（和好例子完全一样）

仍然用这 6 个点（非常重要：同一数据，不同结果）：

左边一团：
• A: (1,1)

• B: (1,2)
• C: (2,1)

右边一团：
• D: (8,8)
• E: (9,8)
• F: (8,9)

———

2️⃣ 坏初始化（关键点）

这次我们故意把两个初始中心都选在左边：
• \mu_1^{(0)} = (1,1) （A）
• \mu_2^{(0)} = (2,1) （C）

👉 直觉提示给学生：

“两个中心一开始就挤在一起，会发生什么？”

———

3️⃣ 第 1 轮：Assignment（分配）

计算距离（只比大小，用平方距离）

———

点 A = (1,1)
• 到 \mu_1：0
• 到 \mu_2：$(1-2)^{2+(1-1)}2=1$

→ A → 簇 1

———

点 B = (1,2)
• 到 \mu_1：$(0)^{2+(1)}2=1$
• 到 \mu_2：$(1-2)^{2+(2-1)}2=1+1=2$

→ B → 簇 1

———

点 C = (2,1)

• 到 \mu_1：1

• 到 \mu_2：0

→ C → 簇 2

———

点 D = (8,8)

• 到 \mu_1：$(7)^{2+(7)}2=98$

• 到 \mu_2：$(6)^{2+(7)}2=85$

→ D → 簇 2

———

点 E = (9,8)

• 到 \mu_1：113

• 到 \mu_2：98

→ E → 簇 2

———

点 F = (8,9)

• 到 \mu_1：113

• 到 \mu_2：100

→ F → 簇 2

———

❗ 第 1 轮分簇结果（问题出现）

• 簇 1：A(1,1), B(1,2)

• 簇 2：C(2,1), D(8,8), E(9,8), F(8,9)

👉 严重不平衡

• 左边 3 个点被拆开

• 右边 3 个点和 1 个左边点混在一起

———

4️⃣ 第 1 轮：Update（更新中心）

———

更新 \mu_1

簇 1：A, B
• x 均值：\frac{1+1}{2}=1
• y 均值：\frac{1+2}{2}=1.5

$$\mu_1^{(1)} = (1, 1.5)$$

———

更新 \mu_2

簇 2：C, D, E, F
• x 均值：\frac{2+8+9+8}{4}=6.75
• y 均值：\frac{1+8+8+9}{4}=6.5

$$\mu_2^{(1)} = (6.75, 6.5)$$

👉 关键观察点（一定要讲）：

第二个中心被“拖”到了左右两团的中间
—— 这是一个非常差的位置

———

5️⃣ 第 2 轮：Assignment（灾难继续）

现在中心是：
• \mu_1=(1,1.5)
• \mu_2=(6.75,6.5)

———

点 C = (2,1)
• 到 \mu_1：(1)$^{2+(-0.5)}$2=1.25
• 到 \mu_2：很大（≈50）

→ C → 簇 1（终于回来了）

———

点 D / E / F

都明显更靠近 \mu_2

———

第 2 轮分簇
• 簇 1：A, B, C
• 簇 2：D, E, F

👉 这次终于分对了

———

6️⃣ 再 Update 一次 → 收敛

更新后：
• \mu_1 \approx (1.33,1.33)
• \mu_2 \approx (8.33,8.33)

和“好初始化”最终结果一致。